

SKILL WEEK5

TASK-1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>
#include <errno.h>

int main() {
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX];
    char path[PATH_MAX];

    previous_dir[0] = '\0';

    printf("==== DIRECTORY NAVIGATION ==== \n");

    while (1) {
        if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
            perror("getcwd");
            return 1;
        }

        printf("\nCurrent Directory: %s\n", current_dir);
        printf("Enter directory path");
        printf(" (.. for parent, - for previous, exit to quit): ");

        if (fgets(path, sizeof(path), stdin) == NULL) {
            break;
        }

        path[strcspn(path, "\n")] = '\0';

        if (strcmp(path, "exit") == 0) {
            printf("Exiting directory navigation.\n");
            break;
        }
    }
}

```

```

        continue;
    }

    char temp[PATH_MAX];
    strcpy(temp, current_dir);

    if (chdir(previous_dir) == -1) {
        perror("chdir");
        continue;
    }

    strcpy(previous_dir, temp);

    printf("Changed to previous directory.\n");
    continue;
}

if (chdir(path) == -1) {
    printf("Error: Cannot change directory to '%s'\n", path);
    perror("Reason");
    continue;
}

strcpy(previous_dir, current_dir);

if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
    perror("getcwd");
    return 1;
}

printf("Directory changed successfully.\n");
printf("New Working Directory: %s\n", current_dir);
}

return 0;
}

```

```
laasya@laasya: ~/skillweek5
laasya@laasya:~$ mkdir -p skillweek5
laasya@laasya:~$ cd skillweek5
laasya@laasya:~/skillweek5$ nano task1.c
laasya@laasya:~/skillweek5$ gcc task1.c -o task1
laasya@laasya:~/skillweek5$ ls
task1 task1.c task1_single_quotes.sh task2_double_quotes.sh
laasya@laasya:~/skillweek5$ ./task1
===== DIRECTORY NAVIGATION =====

Current Directory: /home/laasya/skillweek5
Enter directory path (.. for parent, - for previous, exit to quit): ..
Directory changed successfully.
New Working Directory: /home/laasya

Current Directory: /home/laasya
Enter directory path (.. for parent, - for previous, exit to quit): skillweek5
Directory changed successfully.
New Working Directory: /home/laasya/skillweek5

Current Directory: /home/laasya/skillweek5
Enter directory path (.. for parent, - for previous, exit to quit): -
Changed to previous directory.

Current Directory: /home/laasya
Enter directory path (.. for parent, - for previous, exit to quit): abc123
Error: Cannot change directory to 'abc123'
Reason: No such file or directory

Current Directory: /home/laasya
Enter directory path (.. for parent, - for previous, exit to quit): /tmp
Directory changed successfully.
New Working Directory: /tmp

Current Directory: /tmp
Enter directory path (.. for parent, - for previous, exit to quit): exit
Exiting directory navigation.
laasya@laasya:~/skillweek5$ |
```

TASK-2: TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

/* Function declarations */
void builtin_pwd(char **args);
void builtin_cd(char **args);
void builtin_exit(char **args);
void builtin_help(char **args);

/* Built-in command structure */
typedef struct {
    char *name;
    void (*function)(char **args);
} Builtin;

/* Dispatch table */
Builtin builtins[] = {
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"exit", builtin_exit},
    {"help", builtin_help}
};

int builtin_count = sizeof(builtins) / sizeof(builtins[0]);

/* pwd command */
void builtin_pwd(char **args) {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    }
}

```

```

    printf("\nshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        continue;
    }

    /* Split input into arguments */
    char *token = strtok(input, " ");

    while (token != NULL && argc < MAX_ARGS - 1) {
        args[argc++] = token;
        token = strtok(NULL, " ");
    }

    args[argc] = NULL;

    /* Find built-in command */
    int index = find_builtin(args[0]);

    if (index != -1) {
        /* Dispatch table execution */
        builtins[index].function(args);
    } else {
        /* Execute external command */
        execute_external(args);
    }
}

return 0;
}

```

```

laasya@laasya:~/skillweek5$ nano task2.c
laasya@laasya:~/skillweek5$ gcc task2.c -o task2
laasya@laasya:~/skillweek5$ ./task2
=====
      SIMPLE DISPATCH SHELL
=====
Type 'help' to see built-in commands.

shell> help

Built-in commands:
  pwd    - Display current directory
  cd     - Change directory
  help   - Display available commands
  exit   - Exit the shell

shell> pwd
/home/laasya/skillweek5

```

TASK-3: To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

#define MAX_PATH 4096

/* Save the current directory state */
void save_state(const char *path) {
    FILE *file = fopen("directory_state.txt", "w");

    if (file == NULL) {
        perror("Error opening state file");
        return;
    }

    fprintf(file, "%s\n", path);

    fclose(file);

    printf("Directory state saved successfully.\n");
}

/* Display the current directory */
void display_directory() {
    char current_path[MAX_PATH];

    if (getcwd(current_path, sizeof(current_path)) == NULL) {
        perror("Error retrieving current directory");
        return;
    }

    printf("\nCurrent Directory Path:\n");
    printf("%s\n", current_path);

    save_state(current_path);
}
```

```

        printf("\nInput ended.\n");
        cleanup();
        break;
    }

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "pwd") == 0) {
        display_directory();
    }

    else if (strcmp(command, "save") == 0) {
        char current_path[MAX_PATH];

        if (getcwd(current_path, sizeof(current_path)) == NULL) {
            perror("Error retrieving directory");
        } else {
            save_state(current_path);
        }
    }

    else if (strcmp(command, "exit") == 0) {
        process_exit();
    }

    else if (strlen(command) == 0) {
        printf("Please enter a command.\n");
    }

    else {
        printf("Invalid command: %s\n", command);
    }
}

return 0;
}

```

```

laasya@laasya:~/skillweek5$ nano task3.c
laasya@laasya:~/skillweek5$ gcc task3.c -o task3
laasya@laasya:~/skillweek5$ ./task3
=====
          DIRECTORY STATE MANAGER
=====

Commands:
  pwd  - Display current directory
  save - Save current directory state
  exit - Exit the program

Enter command: pwd

Current Directory Path:
/home/laasya/skillweek5
Directory state saved successfully.

Commands:
  pwd  - Display current directory
  save - Save current directory state
  exit - Exit the program

Enter command: exit

Exit request received.

Cleaning up resources...
Resources cleaned up successfully.
Program terminated safely.
laasya@laasya:~/skillweek5$ |

```

TASK-4: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024

/* Validate environment variable name */
int valid_name(const char *name) {
    int i;

    if (name == NULL || name[0] == '\0') {
        return 0;
    }

    /* First character must be alphabetic or underscore */
    if (!isalpha((unsigned char)name[0]) && name[0] != '_') {
        return 0;
    }

    /* Remaining characters can be alphabetic, digits or underscore */
    for (i = 1; name[i] != '\0'; i++) {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_') {
            return 0;
        }
    }

    return 1;
}

/* Process export command */
void process_export(char *input) {
    char *assignment;
    char *name;
    char *value;

```

```

    process_export(export_data);
}

/* Show command */
else if (strncmp(input, "show ", 5) == 0) {
    char *name = input + 5;

    if (!valid_name(name)) {
        printf("Error: Invalid variable name.\n");
        continue;
    }

    show_variable(name);
}

/* Child process test */
else if (strncmp(input, "child ", 6) == 0) {
    char *name = input + 6;

    if (!valid_name(name)) {
        printf("Error: Invalid variable name.\n");
        continue;
    }

    test_child_environment(name);
}

/* Invalid command */
else {
    printf("Invalid command.\n");
}
}

return 0;
}

```

```
laasya@laasya:~/skillweek5$ nano task4.c
laasya@laasya:~/skillweek5$ gcc task4.c -o task4
laasya@laasya:~/skillweek5$ ./task4
```

```
=====
ENVIRONMENT EXPORT SHELL
=====
```

Commands:

- export NAME=value - Set/update variable
- show NAME - Display variable
- child NAME - Test child inheritance
- exit - Exit program

```
export-shell> export STUDENT=Laasya
Environment variable updated:
STUDENT=Laasya
```

```
export-shell> show STUDENT
STUDENT=Laasya
```

```
export-shell> export STUDENT=ComputerScience
Environment variable updated:
STUDENT=ComputerScience
```

```
export-shell> child STUDENT
```

```
--- Child Process ---
STUDENT=ComputerScience
--- Parent Process ---
Child process completed.
```

```
export-shell> |
```